

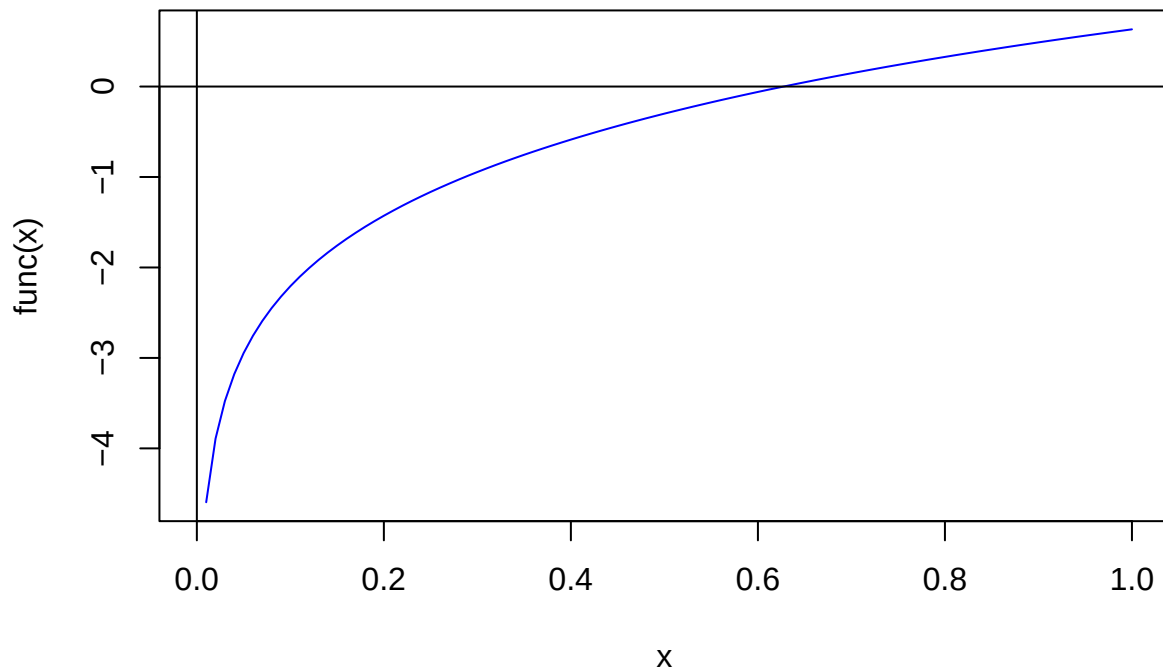
433-HW4

Alex Caramanico

September 27, 2022

Problem 1

```
secant_method = function(f, x0, x1, tol=1e-8, n=1){  
  # parameters: function f(x), initial values, tolerance  
  fval1 = f(x0) # functional value at left point  
  fval2 = f(x1) # functional value at right point  
  f_diff = abs(fval2 - fval1) # difference between functional values  
  x_diff = abs(x1 - x0) # difference between starting x values  
  x_n = x1-fval2*(x_diff/f_diff) # algorithm for next term  
  root_checker = f(x_n) # variable to check functional value as it approaches 0  
  while (abs(root_checker) > tol && n <=100) { # no more than 100 loops  
    x0 = x1 # update x-values  
    x1 = x_n  
    fval1 = f(x0) # update previous values  
    fval2 = f(x1)  
    f_diff = abs(fval2 - fval1) # update differences  
    x_diff = abs(x1 - x0) # SHOULD THIS BE x0 - x_n?  
    x_n = x1-fval2*(x_diff/f_diff) # get next x_value  
    root_checker = f(x_n) # update value of the root checker  
    n = n + 1 # increment steps  
  }  
  return(x_n)  
}  
  
func = function(x) {log(x)-exp(-x)+1} # f(x)  
  
curve(func,from=0,to=1,col='blue') # graph of f(x) from x=0 to x=1  
abline(h=0,col='black') # display x-axis  
abline(v=0,col='black') # display y-axis
```



```
secant_method(func, 0.7, 0.8) # root should be ~0.6275
```

```
## [1] 0.6274702
```

Problem 2

```
halley_root = function(f, fpr, f2pr, x0, tol=1e-8) {
  # parameters: f(x), f'(x), f''(x), initial value, tolerance
  cat('The initial value is', x0, '\n')
  xn = x0 # set x equal to initial value, updated as algorithm converges
  fval = f(xn) # f value at current x
  root_check = abs(fval) # check if value of function is close enough to 0
  ii = 1 # count number of iterations
  while (root_check > tol) {
    numerator = f(xn) # variable for numerator of algorithm
    denom1 = fpr(xn) # two variables for denominator components
    denom2 = (f(xn)*f2pr(xn))/(2*fpr(xn))
    denominator = denom1 - denom2 # variable for entire denominator
    xn = xn - (numerator/denominator) # algorithm for next value
    cat('At step', ii, ', x is', xn, '\n') # display current step and x value
    root_check = f(xn) # update root checker
    ii = ii + 1 # increment iterations
  }
}
```

```

}
func = function(x) {cos(x)-x} # f(x)
funcprime = function(x) {-sin(x)-1} # f'(x)
funcdprime = function(x) {-cos(x)} # f''(x)

halley_root(func,funcprime,funcdprime, 0.2) # real root should be ~0.739

```

```

## The initial value is 0.2
## At step 1 , x is 0.7140229
## At step 2 , x is 0.7390833
## At step 3 , x is 0.7390851

```

```

# f 2 prime 2 furious

```

Problem 3

```

# mean value theorem, test  $x^3-x^2-2x+1$  from  $a = 0$  to  $b = 3$ 
mvt = function(f, a, b) {
  # parameters: f(x), low/high bound
  fp_o_xstar = as.integer((f(b) - f(a)) / (b-a)) # get f'(x*)
  return (fp_o_xstar)
}

new_raph_mvt = function(f, fprime, x0, a, b, tol = 1e-8) {
  xstar = x0 # initial value
  fval = f(x0) # functional value
  root_check = abs(fval) # value to check closeness to root
  cat('Current x value is', xstar, '\n', 'f(x) =', root_check, '\n')
  while (root_check > tol) {
    # loop until x_value is sufficiently close to root
    xstar = xstar - f(xstar)/fprime(xstar) # update xstar; NR algorithm
    fval = f(xstar) # update functional value
    root_check = abs(fval) # update root checker
    cat('Current x value is', xstar, '\n', 'f(x) =', root_check, '\n')
  }
  if (xstar > a & xstar < b) { # check if determined x* value is within range
    return (xstar)
  }
}

func = function(x) {x^3-x^2-2*x+1} # f(x)

funcprime = function (x) {3*x^2-2*x-2} # f'(x)

funcdprime = function(x) {6*x-2} # f''(x) for use in f'(x) NR-method

informed_funcprime = function(x) { # updated f'(x) based on MVT
  #  $3x^2 - 2x - 2 = 4$ , want to solve for x,  $\rightarrow 3x^2 - 2x - 6 = 0$ 
  3*x^2-2*x-6
}

```

```
fp_xstar = mvt(func,0,3) # calculate  $f'(x^*)$  using mean value theorem
fp_xstar
```

```
## [1] 4
```

```
new_raph_mvt(informed_funcprime,funcdprime,3,0,3) # root within range: ~1.7863
```

```
## Current x value is 3
## f(x) = 15
## Current x value is 2.0625
## f(x) = 2.636719
## Current x value is 1.808358
## f(x) = 0.1937638
## Current x value is 1.786465
## f(x) = 0.001438021
## Current x value is 1.7863
## f(x) = 8.160928e-08
## Current x value is 1.7863
## f(x) = 1.776357e-15
```

```
## [1] 1.7863
```